birdclef-2026

16th Place Solution

Rank	16
Team	goonew
Author	goonew
Topic ID	704689
Version	Original

I would like to thank the organizers for providing this valuable opportunity, everyone who shared knowledge in the discussion forum, and the people who published last year's solutions and notebooks. The following shared work was especially important for my final submission:

- @hideyukizushi's public notebook, `bird26-reproduce-perch-protossm-resssm-inf-train`, specifically the LB 0.926 version.
- @dylanliuofficial 's BirdCLEF 2025 4th-place writeup and code repository, especially the Soft AUC loss.
- @vladimirsydor and @vialactea's BirdCLEF 2025 2nd-place solution, especially the pretrained backbones.

I believe there was a considerable element of luck involved in reaching the gold medal range. Still, last year's writeups helped me a lot, so I decided to leave my own solution notes here as well.

Solution Overview

My final submission was a **three-branch rank ensemble**, followed by hierarchical taxonomy smoothing. This smoothing was commonly used in the 0.950-level public notebooks. The three branches were as follows:

1. **Perch branch**: a modified version of @hideyukizushi 's public notebook, `bird26-reproduce-perch-protossm-resssm-inf-train`.
2. **SED branch 1**: SED models with `EfficientNetV2-S` backbone, pretrained on public BirdCLEF data from 2021-2025.
3. **SED branch 2**: SED models with `eca_nfnet_l0` backbone, trained with Soft AUC loss.

Branches 2 and 3 were themselves ensembles of models trained with multiple random seeds.

LB Progression

#	Configuration	Public LB	Private LB
1	**Perch branch** — direct reproduction of @hideyukizushi's public notebook	**0.926**	**0.921**
2	Perch branch v2: changed ProtoSSM objective + post-processing	0.932	0.928
3	#2 + a simple SED trained only on `train_audio` and `labeled soundscapes`	0.934	0.930
4	#3 + SED with source-balanced Focal BCE and soft labels for `train_audio`	0.943	0.940
5	#4 + hard negatives	0.945	0.940
6	#5 + pseudo labels from self-distillation and background-bed augmentation (**deployed SED branch 1**)	0.951	0.946
7	#6 + NFNet trained with Soft AUC (**deployed SED branch 2**)	**0.95586**	**0.95482**

Ensemble and Submission Strategy

After improving the baseline and obtaining a Perch + ProtoSSM pipeline with LB 0.932, I spent some time trying Perch distillation and fine-tuning. However, I found that even simple SED models with lower standalone LB scores could improve the final score when ensembled with the Perch + ProtoSSM pipeline in class-wise percentile/rank space.

Based on that observation, I built many SED models that did not use Perch at all, in order to maximize diversity. I then greedily added models that improved the Public LB. To reduce overfitting to the Public LB as much as possible, each added model was an ensemble across multiple seeds. I also trained additional variants after excluding specific sites, such as S22, from the training data. I submitted these variants to observe how the score changed, and then selected models that performed well on average.

To prioritize which candidates to submit during greedy search, I computed Spearman's rank correlation between model predictions on the validation labeled soundscapes. If adding a model substantially improved the Public LB, I raised the priority of similar models. If it did not help, I lowered the priority of similar models. To save submissions, I eventually stopped measuring the standalone score of each model.

The two SED branches described above were selected through this search process.

Perch Branch

This branch was directly based on @hideyukizushi's public notebook. I reused the core of that notebook without major changes. The referenced version had LB **0.926**.

- **Perch v2**: `google/bird-vocalization-classifier`, executed with ONNX on CPU. It produces frozen embeddings and raw logits for 5-second windows over 234 classes.
- **ProtoSSM v2**: a small state-space model over the sequence of window embeddings: `d_model = 320`, `d_state = 32`, 4 SSM layers, 2 prototypes per class, and 8-head cross-attention. It has **5.78M parameters**.
- **MLP probes**: PCA → 128 → MLP(256, 128), L2 = 0.005, trained for 58 classes.
- **ResidualSSM**: a second SSM that predicts a correction term, with `d_model = 128` and correction weight **0.35**.
- **Within-branch ensemble**: `0.5 × ProtoSSM + 0.5 × MLP + 0.35 × ResidualSSM correction`, with temporal-shift TTA `[0, +1, -1, +2, -2]`.

On top of this setup, I made a few small changes when training ProtoSSM.

1. **Mapped-only distillation**
   I distilled Perch logits only for the 205 of 234 classes that Perch can actually represent. For the remaining classes, the distillation loss was set to zero. My reasoning was that forcing the student to imitate Perch for classes that Perch cannot output would only inject noise.
2. **Pairwise-ranking loss term**
   I added a class-wise hinge loss over sampled positive/negative window pairs: `relu(margin - (s_pos - s_neg))`. The weight was 0.05 and the margin was 1.0. Since the evaluation metric is macro AUC, this was an attempt to directly improve the within-class ordering of predictions.

After adding several small post-processing steps, many of which were also used in public notebooks, this branch improved from LB 0.926 to 0.932.

SED Branch 1

This branch used `tf_efficientnetv2_s.in21k_ft_in1k` as the backbone. It scored 10-second segments with a 5-second hop, using an attention-based temporal pooling head that produced both clip-level and frame-level predictions. Mel-spectrogram settings:

- `n_mels = 224`
- `n_fft = 2048`
- `hop = 512`
- `fmin = 50`
- `fmax = 14000`

The model was trained with Focal BCE.

This branch was built in four steps:

- **Step 1**: pretrain on BirdCLEF competition data from 2021-2025.
- **Step 2**: train using only labeled soundscapes and `train_audio`.
- **Step 3**: use the Step 2 model to add online soft labels to random windows from `train_audio`, then train again.
- **Step 4**: use the Step 3 model to add pseudo labels to unlabeled soundscapes, then train again.

The overall recipe — EfficientNetV2-S, attention-pooling SED head, Focal BCE, EMA, and MixUp — is a fairly standard BirdCLEF SED pipeline. The idea of adding soft labels to `train_audio` was based on last year's 5th-place solution.

My main modifications were the following three points.

(a) Source-Balanced Focal BCE

I assumed that the 5-second labels assigned to soundscape windows were more reliable than the weak labels in `train_audio`, because `train_audio` recordings can be much longer and have only clip-level labels. Therefore, I computed the Focal BCE mean separately for each source and added the two losses:

```
loss_ta = focal(logits[ta_mask], targets[ta_mask]).mean()  # mean over train_audio only
loss_sc = focal(logits[sc_mask], targets[sc_mask]).mean()  # mean over soundscape only
total = loss_ta + loss_sc
```

This makes the two sources contribute equally to the loss, regardless of how many samples from each source are present in the batch.

I tried several samplers, but eventually fixed the batch composition to 75% `train_audio` and 25% `soundscape`. Under that sampler, this loss is equivalent to applying a 3× weight to soundscape samples in Focal BCE.

(b) Weighting Hard Negatives

Each 5-second window has 234 classes, and most class labels are negative. The majority of these negatives are easy: species from completely different habitats that are clearly absent. However, some negatives are hard: species that are plausible for the recording context but are not present in that particular window.

The model tended to produce false positives for these plausible-but-absent species, so I wanted to penalize them more strongly. I tried several variants and eventually used a simple weighting scheme:

- **File-internal hard negatives**: species that are positive elsewhere in the same recording but absent in the current window → **w = 3.0**
- **File-external hard negatives**: species that occur in the same `(site, hour)` group but not in the current file → **w = 2.0**
- **All other negatives** → **w = 1.0**

These weights were multiplied into the per-element focal loss.

(c) Background-Bed Augmentation

`train_audio` clips are often focal recordings: high-SNR and directional, usually targeting a specific individual. Soundscape recordings, on the other hand, are PAM recordings: non-directional and likely to capture sounds from farther away. To reduce this domain gap, I mined 5-second background beds from unlabeled soundscapes and mixed them into each `train_audio` clip with probability **p = 0.1**, after a 2-epoch warm-up.

Before mixing, I applied the following transformations to the `train_audio` waveform.

Distance Simulation

- high-frequency attenuation: uniform(0, 6) dB.
- Blend with a first-order Butterworth low-pass filter, with cutoff sampled from uniform(3000, 6000) Hz.

The goal was to roughly simulate atmospheric absorption, where high frequencies attenuate more strongly with distance.

Reverb

- Add 3-8 early reflections.
- Each reflection has delay `uniform(2, max_delay)` ms, where `max_delay` is clipped to 5-80 ms.
- Each reflection has decay `uniform(0.1, 0.4) × (1 - i / n)`, so later reflections are weaker.
- The delayed copies are added with `result[delay:] += wave[:-delay] * decay`.
- Finally, the waveform is normalized back to the original peak.

The goal was to approximate natural reflections from trees and terrain.

Spectral EQ Transfer

- Estimate the spectral envelopes of both the signal and the background bed using 8 bands.
- Compute `ratio = sqrt(sig_env / bed_env)` and clip it to `[0.3, 3.0]`.

- Apply this gain to the FFT of the background bed, so the bed's timbre is moved closer to that of the signal and does not stand out too much spectrally.
- Smooth the result and apply inverse FFT.

Finally, the augmented waveform was mixed with a random target SNR:

```
snr_db = uniform(5, 20)
target_noise_rms = sig_rms / 10 ** (snr_db / 20)
mixed = wave + bed * (target_noise_rms / noise_rms)
if abs(mixed).max() > 1:
  mixed /= abs(mixed).max()  # peak-normalize only when clipping would occur
```

The SNR range of 5-20 dB was intended to simulate birds at different apparent distances.

This background-bed augmentation was almost LB-neutral when used alone, but applying it to half of the models inside the branch improved the ensemble score. I am not an acoustics expert, so I am not fully confident that all of these transformations are physically correct. It is therefore quite unclear to me whether this augmentation was genuinely beneficial.

When this branch was ensembled with the Perch branch, the score reached LB 0.951.

SED Branch 2

This branch used `eca_nfnet_l0` as the backbone. For inference, each 60-second soundscape was split into six 10-second pieces. The model made frame-wise predictions, and each 5-second window was scored by taking the maximum over the corresponding frames.

It used the same type of attention-based temporal pooling head as Branch 1, producing both clip-level and frame-level predictions.

Mel-spectrogram settings:

- `n_mels = 256`
- `n_fft = 2048`
- `hop = 64`
- `fmin = 60`
- `fmax = 16000`

Instead of Focal BCE, this branch used the AUC surrogate loss, `AUCLoss` / `SoftAUCLoss`, from BirdCLEF 2025 4th-place writeup. I set the labeled-soundscape loss weight to 5.

For pseudo-label generation, I initialized from the large Xeno-Canto pretraining checkpoints released in BirdCLEF 2025 2nd-place solution. I then used an ensemble of models trained with different backbones, mel parameters, and loss functions.

I did not intentionally design this branch to be different from SED Branch 1, but in the end it differed in backbone, scoring procedure, mel parameters, and loss function. This likely helped increase diversity.

Adding this branch finally brought the score to LB 0.955.

I also tried dropping stability within branches: instead of keeping multi-seed ensembles inside each branch, I added more distinct branches. Some of those submissions achieved higher Private LB scores. However, their Public LB scores were at most 0.952, so it was difficult to select them as final submissions.

What Did Not Work

I spent a lot of time building a dedicated sonotype model, but it did not work. I collected insect audio files from external data, then selected only windows whose Perch embeddings were highly similar to the embeddings of sonotype windows. I used those windows to build a pretrained model, and then trained a model targeting only sonotype classes. I tried many parameter settings, but the LB kept getting worse